tf.compat.v1.flags.FlagValues Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/flags/FlagValues

> *Registry of :class:~absl.flags.Flag objects.*

Function Signatures

```
tf.compat.v1.flags.FlagValues()
```

Code Examples

```
tf.compat.v1.flags.FlagValues()
```


```
tf.compat.v1.flags.FlagValues()
```


```
FLAGS['longname'] = x   # register a new flag
```


```
FLAGS['longname'] = x   # register a new flag
```


```
FLAGS.longname  # parsed flag value
FLAGS.x  # parsed flag value (short name)
```


```
FLAGS.longname  # parsed flag value
FLAGS.x  # parsed flag value (short name)
```


```
argv = FLAGS(sys.argv)  # scan command line arguments
```


```
argv = FLAGS(sys.argv)  # scan command line arguments
```


Documentation

Registry of :class:~absl.flags.Flag objects.

A :class:FlagValues can then scan command line arguments, passing flag arguments through to the 'Flag' objects that it owns. It also provides easy access to the flag values. Typically only one

:class:FlagValues object is needed by an application:
:const:FLAGS.

This class is heavily overloaded:

:class:Flag objects are registered via __setitem__::

The .value attribute of the registered :class:~absl.flags.Flag objects can be accessed as attributes of this :class:FlagValues object, through __getattr__. Both the long and short name of the original :class:~absl.flags.Flag objects can be used to access its value::

Command line arguments are scanned and passed to the registered :class:~absl.flags.Flag objects through the __call__ method. Unparsed arguments, including argv[0] (e.g. the program name) are returned::

The original registered :class:~absl.flags.Flag objects can be retrieved through the use of the dictionary-like operator, __getitem__::

The str() operator of a :class:absl.flags.FlagValues object provides help for all of the registered :class:~absl.flags.Flag objects.

Methods

append_flag_values

Appends flags registered in another FlagValues instance.

append_flags_into_file

Appends all flags assignments from this FlagInfo object to a file.

Output will be in the format of a flagfile.

find_module_defining_flag

Return the name of the module defining this flag, or default.

find_module_id_defining_flag

Return the ID of the module defining this flag, or default.

flag_values_dict

Returns a dictionary that maps flag names to flag values.

flags_by_module_dict

Returns the dictionary of module_name -> list of defined flags.

flags_by_module_id_dict

Returns the dictionary of module_id -> list of defined flags.

flags_into_string

Returns a string with the flags assignments from this FlagValues object.

This function ignores flags whose value is None. Each flag
assignment is separated by a newline.

get_flag_value

Returns the value of a flag (if not None) or a default value.

get_flags_for_module

Returns the list of flags defined by a module.

get_help

Returns a help string for all known flags.

get_key_flags_for_module

Returns the list of key flags for a module.

is_gnu_getopt

is_parsed

Returns whether flags were parsed.

key_flags_by_module_dict

Returns the dictionary of module_name -> list of key flags.

main_module_help

Describes the key flags of the main module.

mark_as_parsed

Explicitly marks flags as parsed.

Use this when the caller knows that this FlagValues has been parsed as if
a __call__() invocation has happened. This is only a public method for
use by things like appcommands which do additional command like parsing.

module_help

Describes the key flags of a module.

read_flags_from_files

Processes command line args, but also allow args to be read from file.

This function is called by FLAGS(argv).
It scans the input list for a flag that looks like:
--flagfile=. Then it opens , reads all valid key
and value pairs and inserts them into the input list in exactly the
place where the --flagfile arg is found.

Note that your application's flags are still defined the usual way
using absl.flags DEFINE_flag() type functions.

Notes (assuming we're getting a commandline of some sort as our input):

- For duplicate flags, the last one we hit should "win".
- Since flags that appear later win, a flagfile's settings can be "weak"

if the --flagfile comes at the beginning of the argument sequence,
and it can be "strong" if the --flagfile comes at the end.

- A further "--flagfile=" CAN be nested in a flagfile.

It will be expanded in exactly the spot where it is found.

- In a flagfile, a line beginning with # or // is a comment.
- Entirely blank lines should be ignored.

register_flag_by_module

Records the module that defines a specific flag.

We keep track of which flag is defined by which module so that we can later sort the flags by module.

register_flag_by_module_id

Records the module that defines a specific flag.

register_key_flag_for_module

Specifies that a flag is a key flag for a module.

remove_flag_values

Remove flags that were previously appended from another FlagValues.

set_default

Changes the default value of the named flag object.

The flag's current value is also updated if the flag is currently using the default value, i.e. not specified in the command line, and not set by FLAGS.name = value.

set_gnu_getopt

Sets whether or not to use GNU style scanning.

GNU style allows mixing of flag and non-flag arguments. See http://docs.python.org/library/getopt.html#getopt.gnu_getopt

unparse_flags

Unparses all flags to the point before any FLAGS(argv) was called.

validate_all_flags

Verifies whether all flags pass validation.

write_help_in_xml_format

Outputs flag documentation in XML format.

__call__

Parses flags from argv; stores parsed flags into this FlagValues object.

All unparsed arguments are returned.

__contains__

Returns True if name is a value (flag) in the dict.

__getitem__

Returns the Flag object for the flag --name.

__iter__

__len__